

8장 성능 최적화

⌚ 생성일 @2025년 6월 23일 오후 5:19

1 성능 최적화

1. 데이터를 사용한 성능 최적화
2. 알고리즘을 이용한 성능 최적화
3. 알고리즘 튜닝을 위한 성능 최적화
4. 앙상블을 이용한 성능 최적화

2 하드웨어를 이용한 성능 최적화

1. CPU와 GPU 사용의 차이
2. GPU를 이용한 성능 최적화

3 하이퍼파라미터를 이용한 성능 최적화

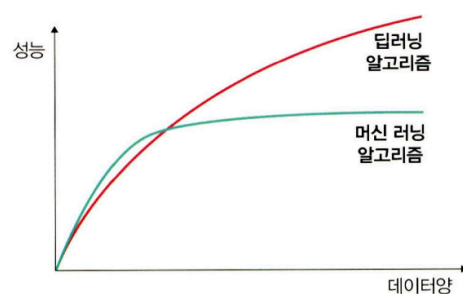
1. 배치 정규화를 이용한 성능 최적화
2. 드롭아웃을 이용한 성능 최적화
3. 조기 종료를 이용한 성능 최적화

1 성능 최적화

1. 데이터를 사용한 성능 최적화

= 많은 데이터를 수집하는 것, but 데이터 수집이 어려운 경우, 임의로 데이터 생성하는 방법 고려

- 최대한 많은 데이터 수집하기: 일반적으로 딥러닝/머신러닝 알고리즘 → 데이터양 ↑ 성능 ↑ ∴ 가능한 많은 데이터(빅데이터)를 수집해야 함



- 데이터 생성하기: 많은 데이터를 수집할 수 없다면 데이터를 만들어 사용 (5장 참고)
- 데이터 범위(scale) 조정하기

- 활성화 함수로 시그모이드 사용 → 데이터 셋 범위를 0~1의 값을 갖도록 / 하이퍼볼릭탄젠트 사용 → 데이터셋 범위를 -1~1의 값을 갖도록 조정
- 정규화, 규제화, 표준화도 성능 향상에 도움

2. 알고리즘을 이용한 성능 최적화

- 유사한 용도의 알고리즘들을 선택 → 모델을 훈련시켜보고 최적의 성능을 보이는 알고리즘 선택
 - 머신러닝 : 데이터 분류를 위해 SVM, K-최근접 이웃 알고리즘들을 선택하여 훈련 및 최종 선택
 - 시계열 데이터 : RNN , LSTM, GRU 등의 알고리즘 선택해 훈련 및 최종 선택

3. 알고리즘 튜닝을 위한 성능 최적화

- 성능 최적화를 하는 데 가장 많은 시간이 소요되는 부분
- 모델을 하나 선택하여 훈련 시키려면 다양한 하이퍼파라미터를 변경하면서 훈련 시키고 최적의 성능을 도출해야 함
- 선택 가능한 파라미터
 - **진단** : 성능 향상이 어느 순간 멈추었을 때 원인을 진단 하는 데 사용할 수 있는것이 모델에 대한 평가
 - 훈련(train) 성능이 검증(test) 보다 눈에 띄게 좋음 → 과적합 의심
⇒ 규제화 진행한다면 성능향상에 도움
 - 훈련과 검증 결과가 모두 성능이 좋지 않음 → 과소적합 의심
⇒ 네트워크 구조를 변경하거나 훈련을 늘리기 위해 에포크수를 조정
 - 훈련 성능이 검증을 넘어서는 변곡점이 있음 → 조기종료 고려
 - **가중치** : 가중치에 대한 초기값은 작은 난수 사용
⇒ 작은 난수라는 숫자가 애매할 경우 → 오토 인코더 같은 비지도학습 이용하여 사전훈련을 진행한 후 지도학습 진행
 - **학습률** : 학습률은 모델의 네트워크 구성에 따라 다르기 때문에 초기에 매우 크거나 작은 임의의 난수를 선택하여 학습 결과를 보고 조금씩 변경해야 함
→ 이때 네트워크의 계층이 많다면 학습률 높게, 네트워크의 계층이 몇 개 되지 않는다면 학습률 작게 설정
 - **활성화 함수** : 활성화 함수의 변경은 신중히
∴ 활성화 함수 변경 시 손실함수도 함께 변경해야 하는 경우가 많음

⇒ 다루고자 하는 데이터 유형 및 데이터로 어떤 결과를 얻고 싶은지 정확하게 이해하지 못했다면 활성화 함수의 변경은 신중해야함

일반적으로는 활성화 함수로 시그모이드나 하이퍼볼릭탄젠트를 사용했다면 출력층에서는 소프트맥스나 시그모이드함수를 많이 선택

- **배치와 에포크** : 일반적으로 큰 에포크와 작은 배치를 사용하는 것이 최근 딥러닝의 트렌드

but, 적절한 배치 크기를 위해 훈련 데이터셋의 크기와 동일하게 하거나 하나의 배치로 훈련을 시켜보는 등 다양한 테스트 진행하는 것이 좋음

- **옵티마이저 및 손실 함수** : 일반적으로 옵티마이저는 확률적경사하강법을 많이 사용함 → 네트워크 구성에 따라 차이는있지만 아담(Adam)이나 알엠에스프롭(RMSProp) 등도좋은성능 보임.

but 이것 역시 다양한 옵티마이저와 손실 함수를 적용해보고성능이 최고인 것을 선택해야 함

- **네트워크 구성** (= 네트워크 토폴로지) : 네트워크 구성을 변경해 가면서 성능 테스트 필요

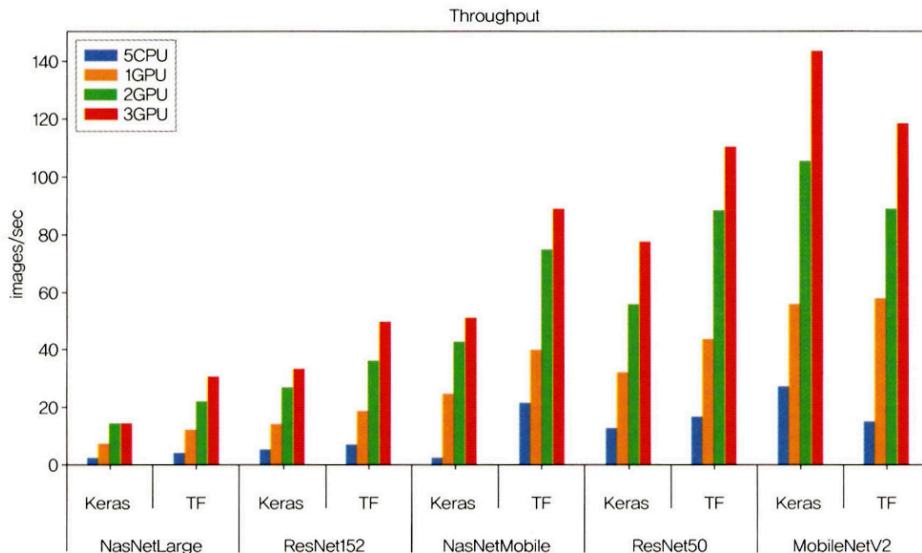
→ ex. 하나의 은닉층에 뉴런을 여러개 포함시키거나(네트워크가 넓다고 표현), 네트워크 계층을 늘리되 뉴런 개수는 줄여봄(네트워크가 깊다고 표현). or 두 가지를 결합

4. 앙상블을 이용한 성능 최적화

- 앙상블 : 모델을 두 개 이상 섞어서 사용

2 하드웨어를 이용한 성능 최적화

1. CPU와 GPU 사용의 차이



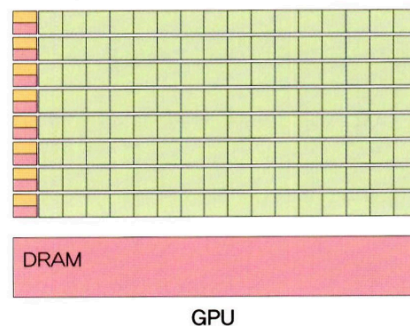
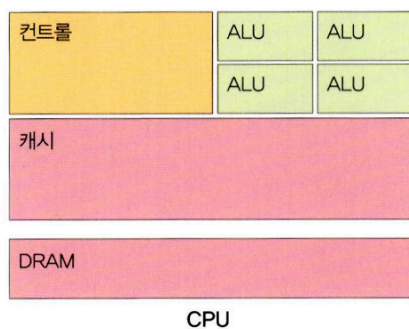
→ CPU 5개를 돌려도 GPU 한 개보다 성능이 좋지 못함

∴ CPU와 GPU는 개발된 목적이 다르고, 그에 따라 내부 구조도 다름

- CPU : 연산을 담당하는 ALU + 명령어를 해석하고 실행하는 컨트롤 + 데이터를 담아두는 캐시
 - 명령어가 입력되는 순서대로 데이터를 처리하는 직렬 처리 방식
 - 한번에 하나의 명령어만 처리 ∴ 연산을 담당하는 ALU (산술논리 장치) 개수가 많을 필요 X
- GPU : 병렬 처리를 위해 개발 → 캐시 메모리 비중 ↓, 연산을 수행하는 ALU 개수 ↑
 - 서로 다른 명령어를 동시에 병렬적으로 처리하도록 설계

∴ GPU는 연산을 수행하는 많은 ALU로 구성 → 여러 명령을 동시에 처리하는 병렬 처리 방식에 특화

 - 하나의 코어에 ALU 수백~수천개 장착 → CPU로는 시간이 많이 걸리는 3D 그래픽 작업 등을 빠르게 수행
- CPU vs. GPU



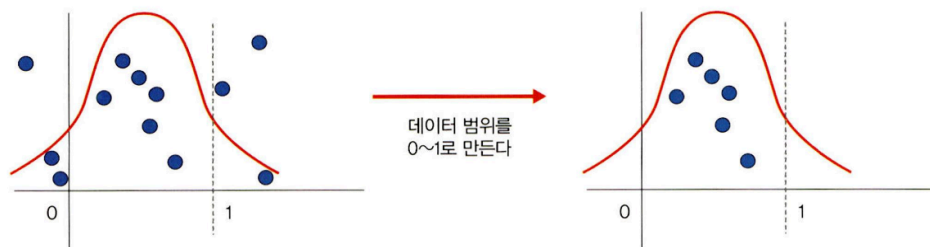
- 개별적 코어 속도: CPU > GPU
- 행렬 연산을 많이 사용하는 재귀 연산 (ex. 파이썬, 매트랩) : CPU > GPU
- 복잡한 미적분 - 병렬 연산 (ex. 역전파) : CPU < GPU
- 딥러닝 : CPU < GPU

2. GPU를 이용한 성능 최적화

3 하이퍼파라미터를 이용한 성능 최적화

1. 배치 정규화를 이용한 성능 최적화

- 정규화 : 데이터 범위를 사용자가 원하는 범위로 제한하는 것

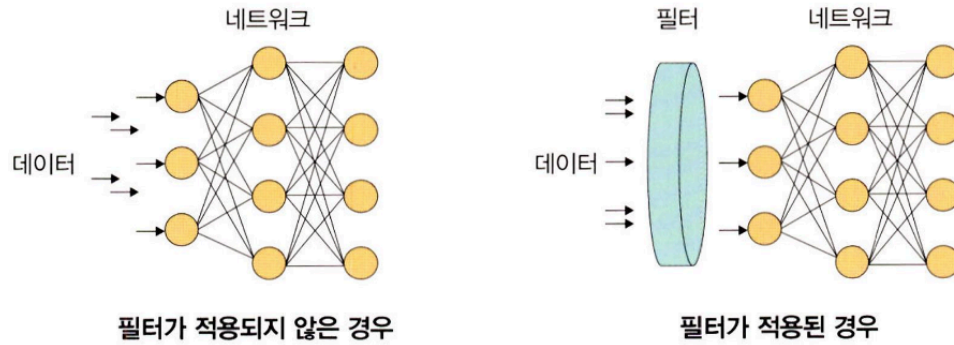


- ex. 이미지 데이터 : 픽셀 정보로 0~255사이의 값을 갖는데, 이를 255로 나누면 0~1.0 사이의 값을 갖게 됨
- 각 특성범위(스케일)를 조정한다는 의미로 특성 스케일이라고도 함
- 스케일 조정을 위해 `MinMaxScaler()` 기법 사용

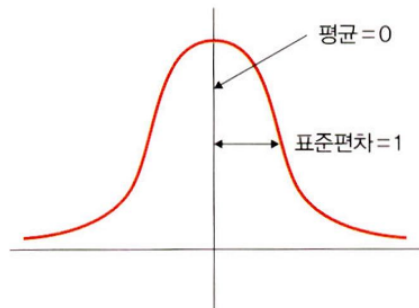
$$\frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

(x: 입력 데이터)

- 규제화 : 모델 복잡도를 줄이기위해 제약을 두는 방법



- 제약 : 데이터가 네트워크에 들어가기 전에 필터를 적용한 것
- [그림] 필터가 적용 되지 않을 경우 모든 데이터가 네트워크에 투입 but, 필터로 걸러진 데이터만 네트워크에 투입되어 빠르고 정확한 결과를 얻을 수 있음
- 규제화 이용해서 모델 복잡도 줄이는 방법 : 드롭아웃 / 조기 종료
- **표준화** : (= 표준화 스칼라, z-스코어 정규화) 기존 데이터를 평균은 0, 표준편차는 1인 형태의 데이터로 만드는 방법

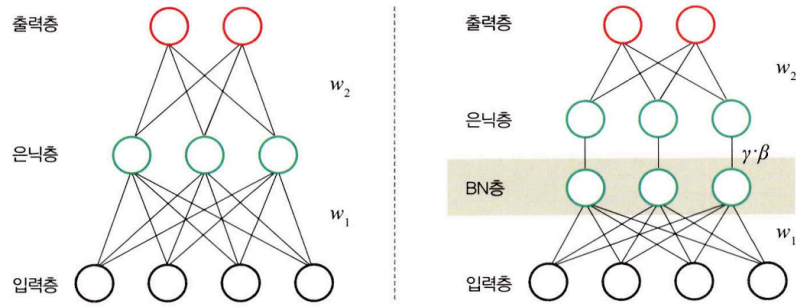


- 평균을 기준으로 얼마나 떨어져 있는지 살펴볼 때 사용
- 보통 데이터 분포가 가우시안 분포를 따를 때 유용한 방법
- 수식

$$\frac{x - m}{\sigma}$$

(σ : 표준편차, x : 관측 값(입력 값), m : 평균)

- **배치 정규화** : 데이터 분포 안정화 → 학습속도 ↑

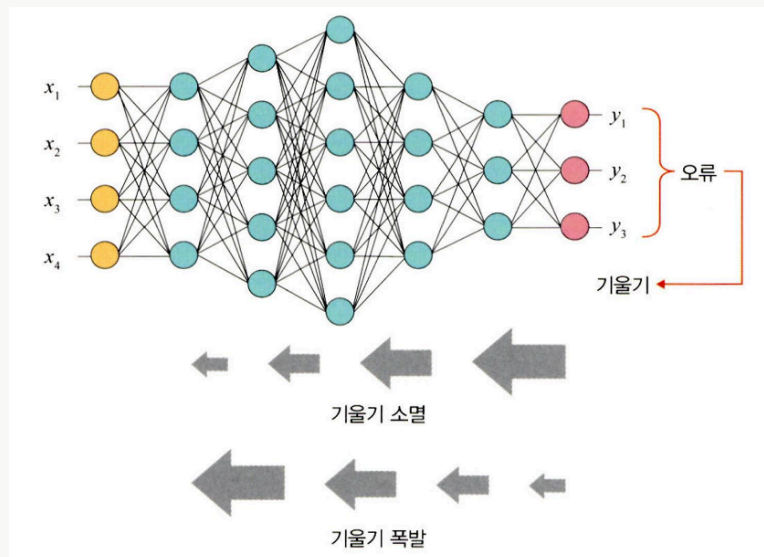


- 기울기 소멸·폭발 같은 문제 해결하기 위한 방법 → 손실 함수로 렐루(ReLU)를 사용하거나 초깃값튜닝, 학습률 등을 조정



기울기 소멸과 기울기 폭발

- 기울기 소멸: 오차 정보를 역전파 시키는 과정에서 기울기가 급격히 0에 가까워져 학습이 되지 않는 현상
- 기울기 폭발: 학습 과정에서 기울기가 급격히 커지는 현상



- 기울기 소멸·폭발 원인 : 내부 공변량 변화 = 네트워크의 각 층마다 활성화 함수가 적용 되면서 입력값들의 분포가 계속 바뀌는 현상
→ 분산된 분포를 정규분포로 만들기 위해 표준화와 유사한 방식을 미니배치에 적용하여 평균은 0으로, 표준편차는 1로 유지 하도록 함

$$\mu\beta \leftarrow \frac{1}{m} \sum_{i=1}^m x_i \quad \text{----①}$$

$$\sigma^2\beta \leftarrow \frac{1}{m} \sum_{i=1}^m (x_i - \mu\beta)^2 \quad \text{----②}$$

$$\hat{x}_i \leftarrow \frac{x_i - \mu\beta}{\sqrt{\sigma^2\beta + \epsilon}} \quad \text{----③}$$

$$y_i \leftarrow \gamma \hat{x}_i + \beta \Leftrightarrow BN_{\gamma, \beta}(x_i) \quad \text{----④}$$

$$\left(\begin{array}{l} \text{입력: } \beta = \{x_1, x_2, \dots, x_n\} \\ \text{학습해야 할 하이퍼파라미터: } \gamma, \beta \\ \text{출력: } y_i = BN_{\gamma, \beta}(x_i) \end{array} \right)$$

1. 미니 배치 평균 구함
2. 미니 배치의 분산과 표준편차 구함
3. 정규화 수행
4. 스케일 조정 (= 데이터 분포 조정)

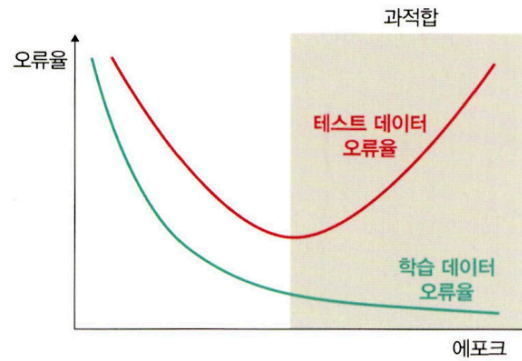
⇒ 매 단계마다 활성화 함수를 거치면서 데이터셋 분포가 일정 → 속도 ↑ but 단점 존재

- 배치 크기 ↓ : 정규화 값이 기존값과 다른 방향으로 훈련될 수 있음
ex. 분산 = 0, 정규화 자체가 안됨
- RNN은 네트워크 계층 별로 미니 정규화 적용해야 함 → 모델이 더 복잡해지면
서 비효율적일수 있음

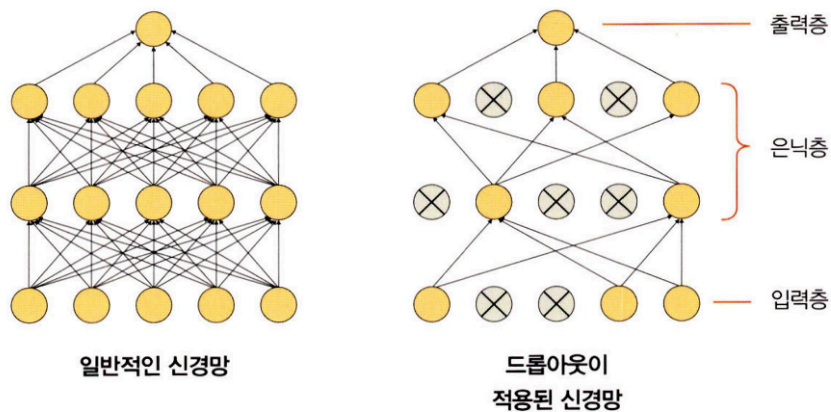
⇒ 가중치 수정, 네트워크 구성 변경 등 수행하여 해결

2. 드롭아웃을 이용한 성능 최적화

- 과적합 : 훈련 데이터셋을 과하게 학습하는 것 → 오류는 줄어들이지만 테스트 데이터셋에 대한 오류는 어느순간부터 증가



- 드롭아웃 : 훈련할 때 일정 비율의 뉴런만 사용하고, 나머지 뉴런에 해당하는 가중치는 업데이트하지 않는 방법
 = 노드를 임의로 끄면서 학습하는 방법으로, 은닉층에 배치된 노드 중 일부를 임의로 끄면서 학습
 → 꺼진노드는 신호를 전달하지 않으므로 지나친 학습을 방지하는 효과



- 일부 노드들은 비활성화 되고 남은 노드들로 신호가 연결되는 신경망 형태
- 어떤 노드를 비활성화 할지는 학습할 때마다 무작위로 선정, 테스트 데이터로 평가 할 때는 노드들 을 모두 사용하여 출력하되 노드 삭제 비율(드롭아웃 비율)을 곱해서 성능 평가
- 훈련 시간이 길어지는 단점 有

코드 8-1 라이브러리 호출

코드 8-2 데이터셋 내려받기

코드 8-3 데이터셋을 메모리로 가져오기

코드 8-4 데이터셋 분리

1

```
torch.Size([4, 1, 28, 28])
```

(a) (b) (c)

- a. 한 번의 배치 크기로 몇 개의 데이터를 가져오는지 의미, 앞에서 `batch_size = 4`를 지정 했기 때문에 4 출력
- b. 채널을 의미, 흑백 이미지 → 1을 출력, 컬러이미지 → 3을 출력
- c. 28×28(너비×높이) 픽셀 크기의 이미지라는 의미

코드 8-5 이미지 데이터를 출력하기 위한 전처리

1

```
plt.imshow(np.transpose(img, (1, 2, 0)))
```

- 파이토치 이미지 데이터셋 → [batch_size, channel, width, height] 순으로 저장
 - matplotlib 출력 → [width, height, channel] 형태
- ∴ 데이터 형테 변경 필요 ⇒ `transpose()` 사용

코드 8-5 이미지 데이터를 출력하기 위한 전처리

코드 8-6 이미지 데이터 출력 함수

코드 8-7 이미지 출력

코드 8-8 배치 정규화가 적용되지 않은 네트워크

코드 8-9 배치 정규화가 포함된 네트워크

1

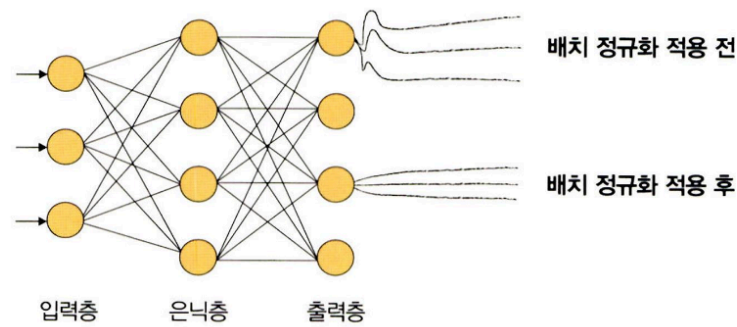
```
nn.BatchNorm1d (48)
```

- 배치 정규화가 적용되는 부분, `BatchNorm1d` 에서 사용되는 파라미터는 특성 개수로 이전 계층의 출력 채널이 됨

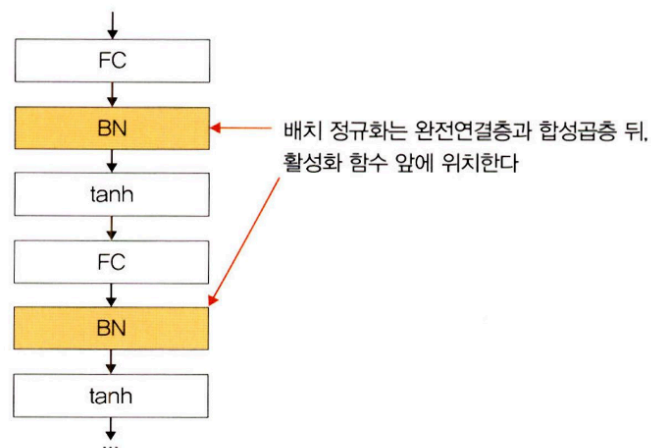
- 배치 정규화 사용 이유 : 은닉층에서 학습이 진행될 때마다 입력분포가 변하면서 가중치가 엉뚱한방향으로 갱신되는 문제가 종종 발생하기 때문

= 신경망의 층이 깊어질수록 학습할 때 가정했던 입력 분포가 변화하여 엉뚱한 학습 진행 가능

⇒ 배치 정규화를 적용해서 입력 분포를 고르게 맞추어 줄 수 있음



- 배치 정규화 위치



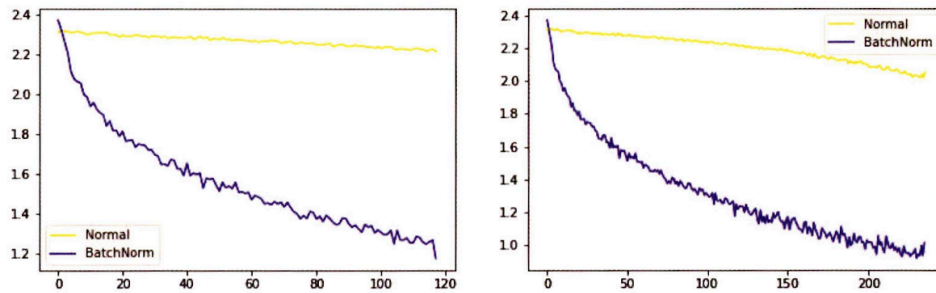
코드 8-10 배치 정규화가 적용되지 않은 모델 선언

코드 8-11 배치 정규화가 적용된 모델 선언

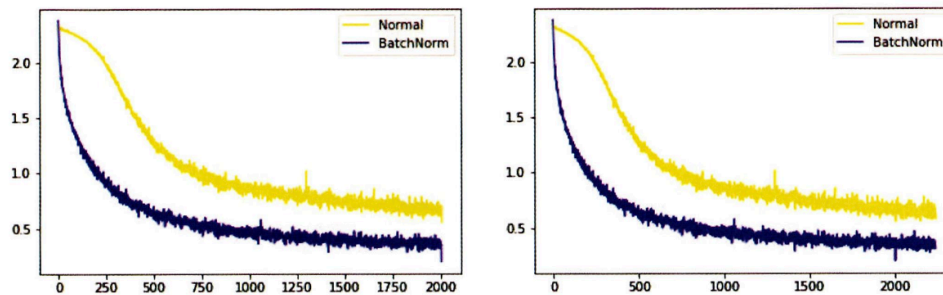
코드 8-12 데이터셋 메모리로 불러오기

코드 8-13 옵티마이저, 손실함수 지정

코드 8-14 모델 학습



... 중간 생략 ...



- 배치 정규화 적용된 모델/미적용 모델 둘 다 시간이 흐를수록 오차가 줄어들 but 오차가 줄어드는 범위 및 값의 차이 존재
- 배치 정규화가 적용된 모델의 경우 더 낮은 값으로 안정적인 범위 내에서 줄어들
= 에포크가 진행 될수록 오차도 줄어들면서 안정적인 학습

코드 8-15 데이터셋의 분포를 출력하기 위한 전처리

1

```
x_train = torch.unsqueeze(torch.linspace(-1, 1, N), 1)
```

(a)

(b)

- 훈련 데이터 셋이 1~1의 값을 갖도록 조정
- a. `torch.linspace` : 주어진 범위에서 균등한 값을 갖는 텐서를 만들기 위해 사용
 → `torch.linspace(-1,1,N)` = -1 ~1의 범위에서 N개의 균등한 값을 갖는 텐서를 생성
- b. `torch.unsqueeze()` : 차원을 늘리기 위해 사용
 → `torch.unsqueeze(torch.linspace(-1, 1,N),1)` = `torch.linspace(-1, 1, N)` 텐서의 첫 번째 자리에 차원을 증가

2

```
y_train = x_train + noise * torch.normal(torch.zeros(N, 1), torch.ones(N, 1))
```

(a)

(b)

(c)

- a. `torch.normal` : 정규분포로부터 무작위 표본 추출을 위해 사용함
 - `torch.normal(평균, 표준편차)`를 의미
 - `torch.zeros()` : 평균, `torch.ones()` : 표준편차
 - 평균은 0, 표준 편차는 1이 기본값
- b. `torch.zeros()` : 0 값을 갖는 Nx1 텐서 생성
- c. `torch.ones()` : 1 값을 갖는 Nx1 텐서 생성

코드 8-16 데이터 분포를 그래프로 출력

1

```
plt.scatter(x_train.data.numpy(), y_train.data.numpy(), c='purple',
            alpha=0.5, label='train')
```

(a) (b) (c)
(d) (e)

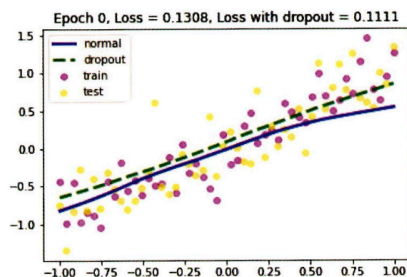
- a. 첫 번째 파라미터: x축에 위치할 데이터
- b. 두 번째 파라미터: y축에 위치할 데이터
- c. `c`: 그래프로 출력되는 마커의 색상
- d. `alpha`: 마커에 대한 투명도를 조절, `alpha=1`이면 완전 불투명 상태
- e. `label`: 맷플롯립의 레전드와 같은 역할을 하지만 `plt.legend()`와 함께 사용되어야 함

코드 8-17 드롭아웃을 위한 모델 생성

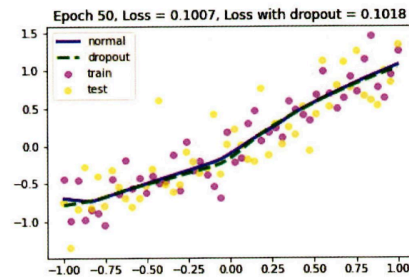
코드 8-18 옵티마이저와 손실함수 지정

코드 8-19 모델 학습

▼ 그림 8-46 첫 번째 드롭아웃에 대한 학습 결과

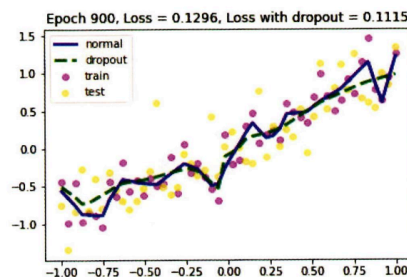


▼ 그림 8-47 두 번째 드롭아웃에 대한 학습 결과

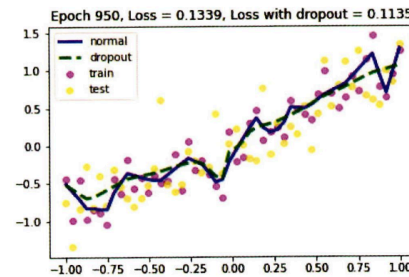


... 중간 생략 ...

▼ 그림 8-48 19번째 드롭아웃에 대한 학습 결과



▼ 그림 8-49 20번째 드롭아웃에 대한 학습 결과

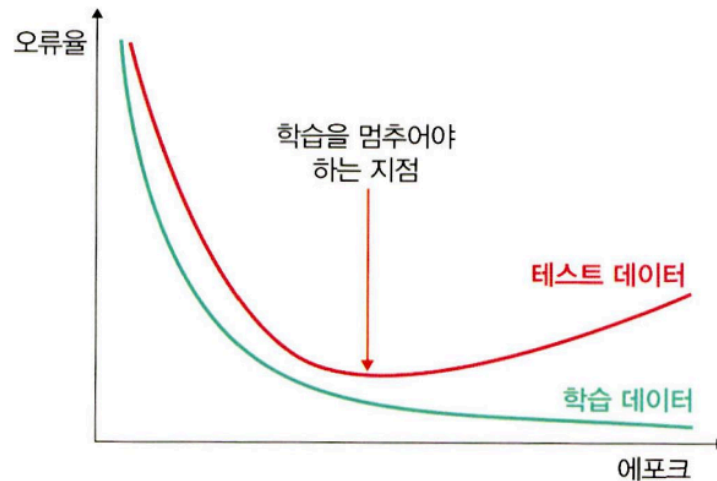


- 전반적으로 오차가 줄어드는 범위는 크지 않음 but 드롭아웃을 적용했을 때의 오차가 더 낮음

- 왼쪽 과적합 발생 : 훈련횟수가 늘어날수록 파란색 실선은 가장자리의 자주색 점 (훈련 데이터) 들을 찾아가고 있음 ↔ 오른쪽 드롭아웃

3. 조기 종료를 이용한 성능 최적화

- 조기 종료 : 뉴럴 네트워크가 과적합을 회피하는 규제 기법



- 훈련 데이터와 별도로 검증 데이터를 준비하고, 매 에포크마다 검증 데이터에 대한 오차를 측정하여 모델의 종료시점을 제어함
 - ⇒ 과적합 발생하기 전 : 학습에 대한 오차와 검증에 대한 오차 모두 감소
 - ↔ 과적합 발생 : 훈련 데이터셋에 대한 오차는 감소하는 반면 검증 데이터셋에 대한 오차는 증가
 - ⇒ 조기 종료: 검증 데이터셋에 대한 오차가 증가하는 시점에서 학습을 멈추도록 조정

코드 8-20 라이브러리 호출

코드 8-21 데이터셋 전처리

코드 8-22 데이터셋 가져오기

코드 8-23 모델 생성

코드 8-24 학습률 감소

1

```
torch.optim.lr_scheduler.ReduceLROnPlateau(self.optimizer, mode='min',
                                             (a)          (b)          (c)
                                             patience=self.patience, factor=self.factor, min_lr=self.min_lr, verbose=True)
                                             (d)          (e)          (f)          (g)
```

학습 과정에서 모델 성능에 대한 개선이 없을 경우 학습률 값을 조절하여 모델의 개선을 유도하는 콜백 함수

- a. `lr_scheduler.ReduceLROnPlateau` : 검증 데이터셋에 대한 오차의 변동이 없으면 학습률을 `factor`배로 감소
- b. `optimizer` : 파라미터(가중치)를 갱신시키는 부분으로, 여기에서는 아담(`optim.Adam`)을 사용
- c. `mode` : 언제 학습률을 조정할 지에 대한 기준이 되는 값
 - 만약 검증 데이터셋에 대한 오차를 기준으로 사용 → 오차가 더 이상 감소되지 않을 때 학습률을 조정
 - 이때 오차값이 최소(min)가 되어야 하는지, 최대(max)가 되어야 하는지 알려주는 파라미터가 `mode`
 - ex. 학습률 조정의 기준이 되는 값을 모델의 정확도(`val_acc`)로 사용 → 값이 클수록 좋기 때문에 'max'를 지정 / 모델의 오차(`val_loss`)로 사용 → 작을수록 좋기 때문에 'min'을 지정
- c. `patience` : 학습률을 업데이트하기 전에 몇 번의 에포크를 기다려야 하는지 결정하는 것으로, 여기에서는 다섯 번의 에포크를 기다리도록 설정
- d. `factor` : 학습률을 얼마나 감소시킬지 지정하는 파라미터

새로운 학습률 = 기존학습률 * `factor`
- e. `min_lr` : 학습률의 하한선을 지정

ex. 현재 학습률 = 0.1, `factor` = 0.5, `min_lr` = 0.03

→ 첫 번째로 콜백함수 적용 시 학습률의 하한선 값 = $0.03 \times (0.1 \times 0.5)$
- f. `verbose` : 조기 종료의 시작과 끝을 출력하기 위해 사용
 - 1로 설정할 경우 : 조기종료가 적용되면 적용되었다고 화면에 출력
 - 0으로 설정할 경우: 아무런 출력없이 학습 종료

2

```
self.lr_scheduler.step(val_loss)
```

실제로 학습률을 업데이트, 에포크 단위로 검증 데이터셋에 대한 오차(val_loss)를 받아서 이전 오차와 비교했을 때 차이가 없다면 학습률을 업데이트



콜백 함수

- 개발자가 명시적으로 함수를 호출하는 것이 아니라 개발자는 단지 함수 등록만 하고 특정 이벤트 발생에 의해 함수를 호출하고 처리하도록 하는 것
- 콜백 함수: 동기적(synchronous) 함수 / 비동기적 (asynchronous) 함수
 - 동기적 함수 : 코드가 위에서 아래로, 왼쪽에서오른쪽으로 순차적으로 실행되는 함수
 - 비동기함수 : 병렬 처리와 같음 , 어떤 코드를 실행했을 때 상당한 시간을 기다려야 하는 경우, 해당 코드가 완료될 때까지 기다리는 것이 아닌 다른 코드가 먼저 처리되도록 하는 것

코드 8-25 조기 종료

1

```
self.patience = patience
```

- **patience** : 오차개선이 없다고 바로 종료하지 않고 개선이 없는 에포크를 얼마나 기다려줄지 지정

2

```
self.delta = delta
```

- **delta** : 오차가 개선되고 있다고 판단하기 위한 최소 변화량, 변화량이 delta보다 적다면 개선이 없다고 판단

- 케라스 콜백

```
from keras.callbacks import ModelCheckpoint, EarlyStopping
checkpoint = ModelCheckpoint('checkpoint-epoch.h5'.format(EPOCH, BATCH_SIZE),
                             monitor='val_loss', ----- val_loss 값이 개선되었을 때 호출
                             verbose=1, ----- 로그 출력
                             save_best_only=True, ----- 가장 최적의 값만 저장
                             mode='auto' ----- auto 의미는 시스템이 알아서 best 값을 찾으라는 것
                             )

earlystopping = EarlyStopping(monitor='val_loss', ----- 학습률 업데이트 기준 설정(val_loss)
                              patience=10 ----- 에포크가 진행되는 열 번 동안 모델의 오차가
                              )                       개선되지 않는다면 종료
```

코드 8-26 인수 값 지정

1

```
parser.add_argument('--lr-scheduler', dest='lr_scheduler', action='store_true')
```

(a) (b) (c)

원하는 인수 값을 추가, `parser.add_argument()` 는 인수 개수 만큼 만들

- 첫번째 파라미터: 옵션 문자열의 이름으로 명령을 실행할 때 사용
- `dest`: 입력 값이 저장되는 변수
- `action`: `store_true` 로 지정하면 입력값을 `dest` 파라미터에 의해 생성된 변수에 저장

2

```
args = vars(parser.parse_args())
```

입력 받은 인수 값이 실제로 `args` 변수에 저장

코드 8-27 사전 훈련된 모델의 파라미터 확인

코드 8-28 옵티마이저와 손실함수 지정

코드 8-29 오차, 정확도 및 모델의 이름에 대한 문자열

코드 8-30 오차, 정확도 및 모델의 이름에 대한 문자열

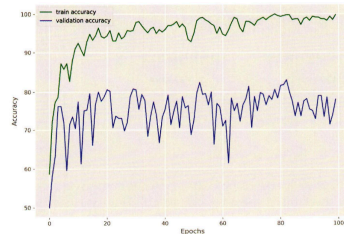
코드 8-31 모델 학습 함수

코드 8-32 모델 검증 함수

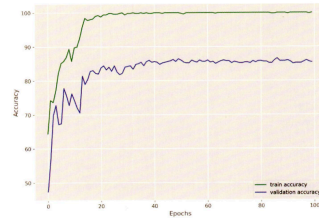
코드 8-33 모델 학습

코드 8-34 모델 학습 결과 출력

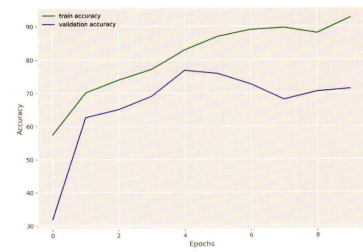
▼ 그림 8-54 어떤 인수도 적용되지 않았을 때의 정확도



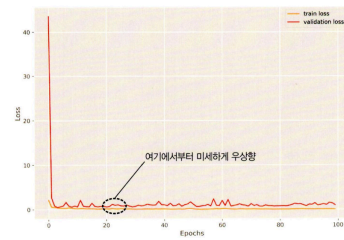
▼ 그림 8-56 학습률 감소에 대한 인수가 적용되었을 때의 정확도



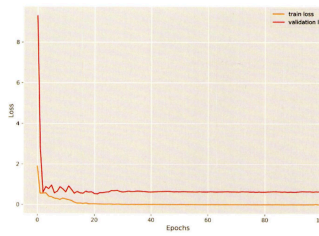
▼ 그림 8-58 초기 종료에 대한 인수가 적용되었을 때의 정확도



▼ 그림 8-55 어떤 인수도 적용되지 않았을 때의 오차



▼ 그림 8-57 학습률 감소에 대한 인수가 적용되었을 때의 오차



▼ 그림 8-59 초기 종료에 대한 인수가 적용되었을 때의 오차

